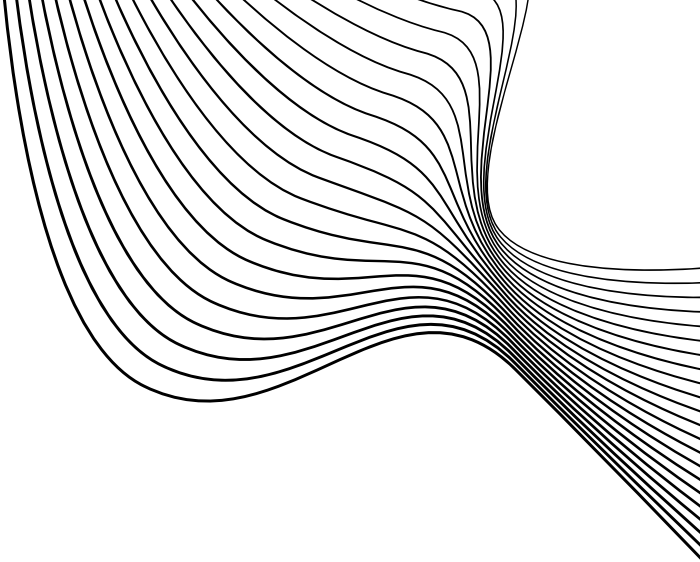
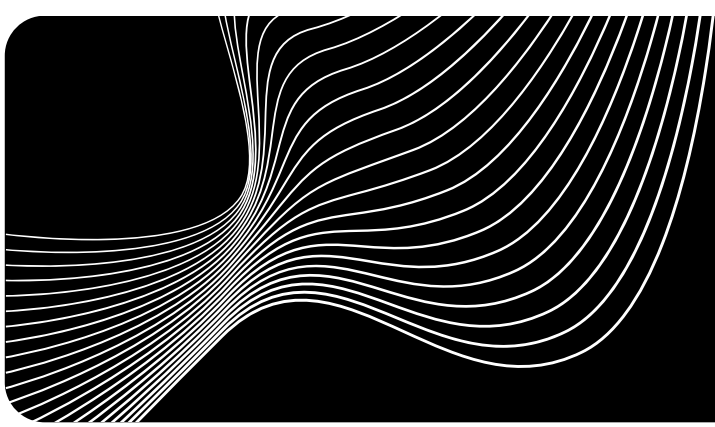




ScripticX

O platformă de programare, creată de elevi
pentru elevi.



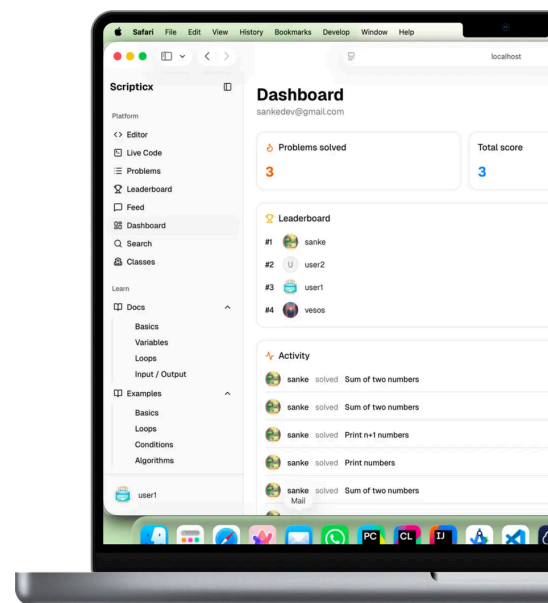
 www.scripticx.org

Colegiul Național "Gheorghe
Munteanu Murgoci"



Cuprins

- I Problema identificată... p.4
- II Scopul și obiectivele proiectului...p. 5-6
- III Context educațional și relevanță...p. 7
- IV Arhitectura platformei ScripticX...p. 8
- V Tehnologii utilizate...p. 9
- VI Limbajul MiniScript+ și engine-ul...p. 10-12
- VII Funcționalitățile platformei...p. 13-14
- VIII Evaluare automată, feedback și analiză de complexitate...p. 15
- IX Elemente inovative...p. 16
- X Testare, securitate și maturitate...p. 17



Cuprins

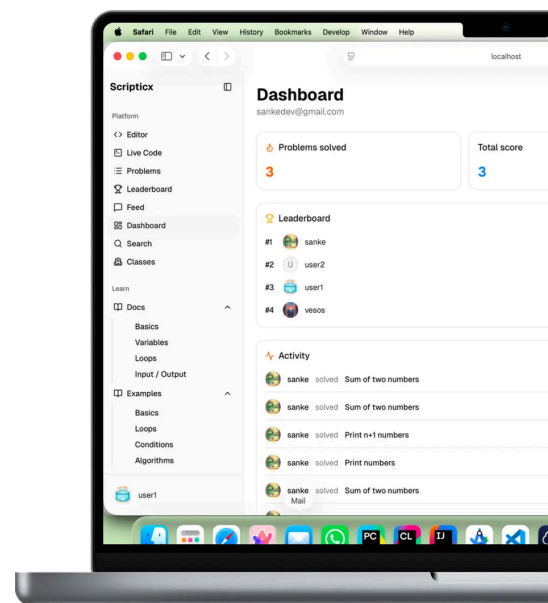
XI Interfața UI/UX...p. 18

XII Etapele de cercetare și dezvoltare...p. 19

XIII Extensii viitoare și direcții de dezvoltare...p. 20

XIV Componenta echipei...p. 21-22

XV Link-uri utile și resurse...p. 23



I. Problema identificată

Învățarea programării pentru începători este dificilă deoarece elevii trebuie să înțeleagă simultan sintaxa limbajului, conceptele algoritmice și modul de execuție al programului. Platformele educaționale existente oferă adesea feedback limitat, concentrat pe verdictul final, fără a explica suficient pașii intermediari ai execuției. În paralel, utilizarea excesivă a instrumentelor automate de generare a codului poate reduce implicarea activă a elevilor în procesul de rezolvare, dacă aceștia primesc soluții finale fără să înțeleagă logica algoritmică.



Sintaxă dificilă

Elevii trebuie să învețe reguli de sintaxă, tipuri de date și structuri de control înainte să poată urmări clar logica algoritmică.



Feedback insuficient

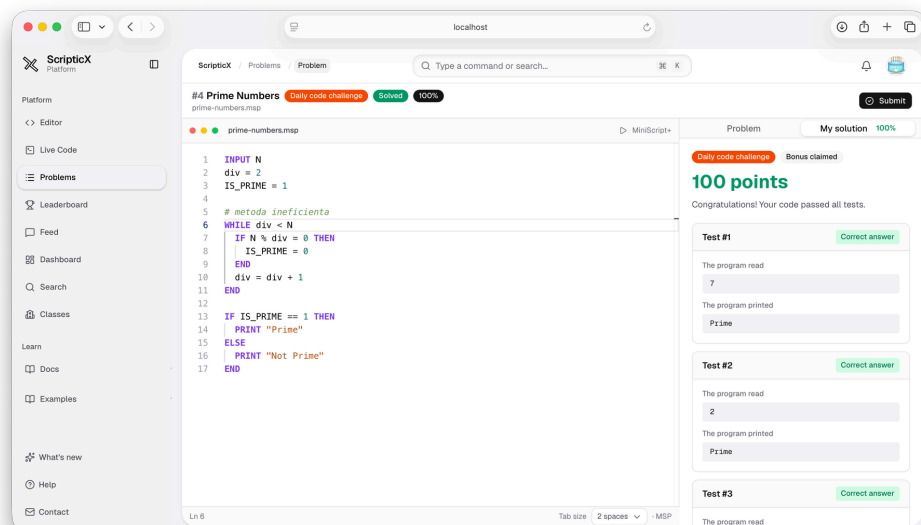
Multe platforme afișează doar dacă soluția este corectă sau greșită, fără să explice pas cu pas unde apare eroarea.



Înțelegere superficială

Soluțiile copiate sau generate automat pot funcționa, dar nu dezvoltă capacitatea elevului de a raționa și optimiza algoritmi.

Această combinație dintre complexitatea inițială a programării, lipsa feedback-ului explicativ și dependența de soluții generate automat evidențiază nevoia unei platforme care să facă procesul de învățare mai vizibil, interactiv și orientat spre înțelegerea logicii din spatele algoritmilor.



II. Scopul și obiectivele proiectului

Învățarea programării pentru începători este dificilă deoarece elevii trebuie să înțeleagă simultan sintaxa limbajului, conceptele algoritmice și modul de execuție al programului. Platformele educaționale existente oferă adesea feedback limitat, concentrat pe verdictul final, fără a explica suficient pașii intermediari ai execuției. În paralel, utilizarea excesivă a instrumentelor automate de generare a codului poate reduce implicarea activă a elevilor în procesul de rezolvare, dacă aceștia primesc soluții finale fără să înțeleagă logica algoritmică.

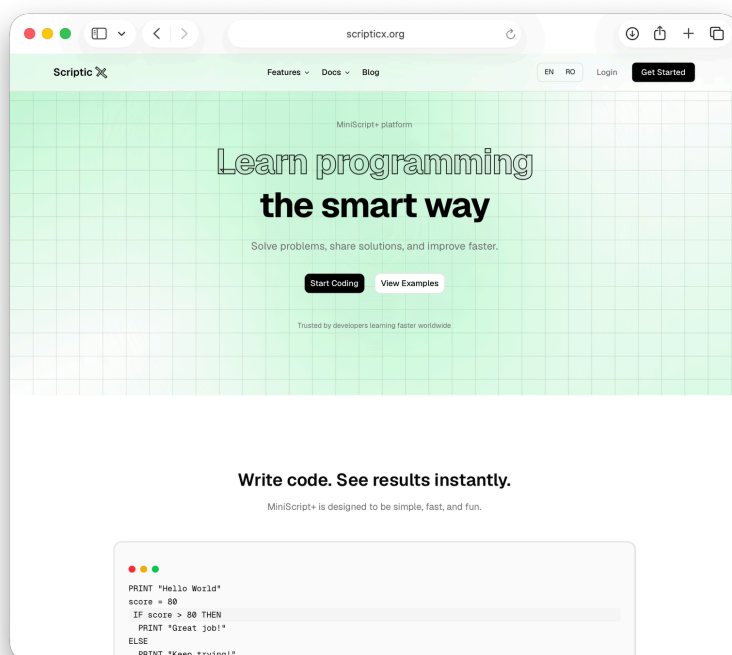
Spre deosebire de platformele educaționale tradiționale, ScripticX propune o experiență interactivă, vizuală și orientată spre înțelegerea procesului, nu doar spre obținerea unui verdict final.

PLATFORME TRADIȚIONALE

- utilizează limbaje complexe (C++/Java)
- mesaje de eroare în engleză
- fără feedback vizual
- Fără analiză de complexitate
- Fără colaborare între utilizatori

ScripticX

- limbaj educațional MiniScript+
- execuție pas cu pas și debugger
- feedback automat pe test case-uri
- analiză Big-O pentru timp și spațiu
- AST, flowchart și colaborare live
- daily challenges, snippets și feed social



II. Scopul și obiectivele proiectului

Obiectivele proiectului ScripticX sunt orientate spre construirea unui mediu educațional complet, în care elevii pot învăța programarea prin experimentare, feedback automat și vizualizarea logicii algoritmice.



Accesibilitate

Reducerea barierei de intrare în programare printr-un limbaj educațional simplificat, apropiat de pseudocod și ușor de înțeles pentru începători.



Feedback Clar

Transformarea procesului de învățare într-o experiență activă, în care elevul scrie cod, rulează, testează și observă rezultatele imediat.



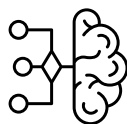
Învățare Interactivă

Oferirea unui feedback automat pe test case-uri, astfel încât utilizatorul să înțeleagă ce funcționează, ce e greșit și unde trebuie îmbunătățită soluția.



Vizualizarea Execuției

Evidențierea pașilor interni ai programului prin debugger, variabile, linie curentă, AST și flowchart, pentru a face logica algoritmică observabilă.



Gândire Algoritmică

Încurajarea optimizării soluțiilor prin analiză de complexitate în timp și spațiu și prin sugestii legate de eficiența algoritmilor.



Colaborare și Continuitate

Susținerea învățării constante prin sesiuni live, snippets multi-fișier, daily code challenges, profiluri, feed social și leaderboard.

III. Context educațional și relevanță

În contextul digitalizării educației, învățarea programării devine o competență esențială pentru dezvoltarea gândirii logice, a creativității și a capacității de rezolvare a problemelor. Totuși, pentru mulți elevi aflați la început, programarea este percepută ca un domeniu dificil, din cauza sintaxei complexe și a lipsei unui feedback suficient de clar.

ScripticX răspunde acestei nevoi printr-o platformă care combină un limbaj educațional simplificat, evaluare automată, vizualizarea execuției și colaborare în timp real. Astfel, elevii pot învăța prin experimentare directă, pot testa ipoteze, pot observa efectele instrucțiunilor și pot înțelege mai bine logica algoritmică.



Relevanță educațională

Platforma sprijină formarea competențelor digitale și algoritmice, oferind elevilor un mediu accesibil pentru învățarea programării prin practică.



Învățare prin experimentare

Elevii pot rula cod, modifica soluții, observa variabilele și înțelege pas cu pas modul în care instrucțiunile influențează rezultatul final.



Impact asupra elevilor

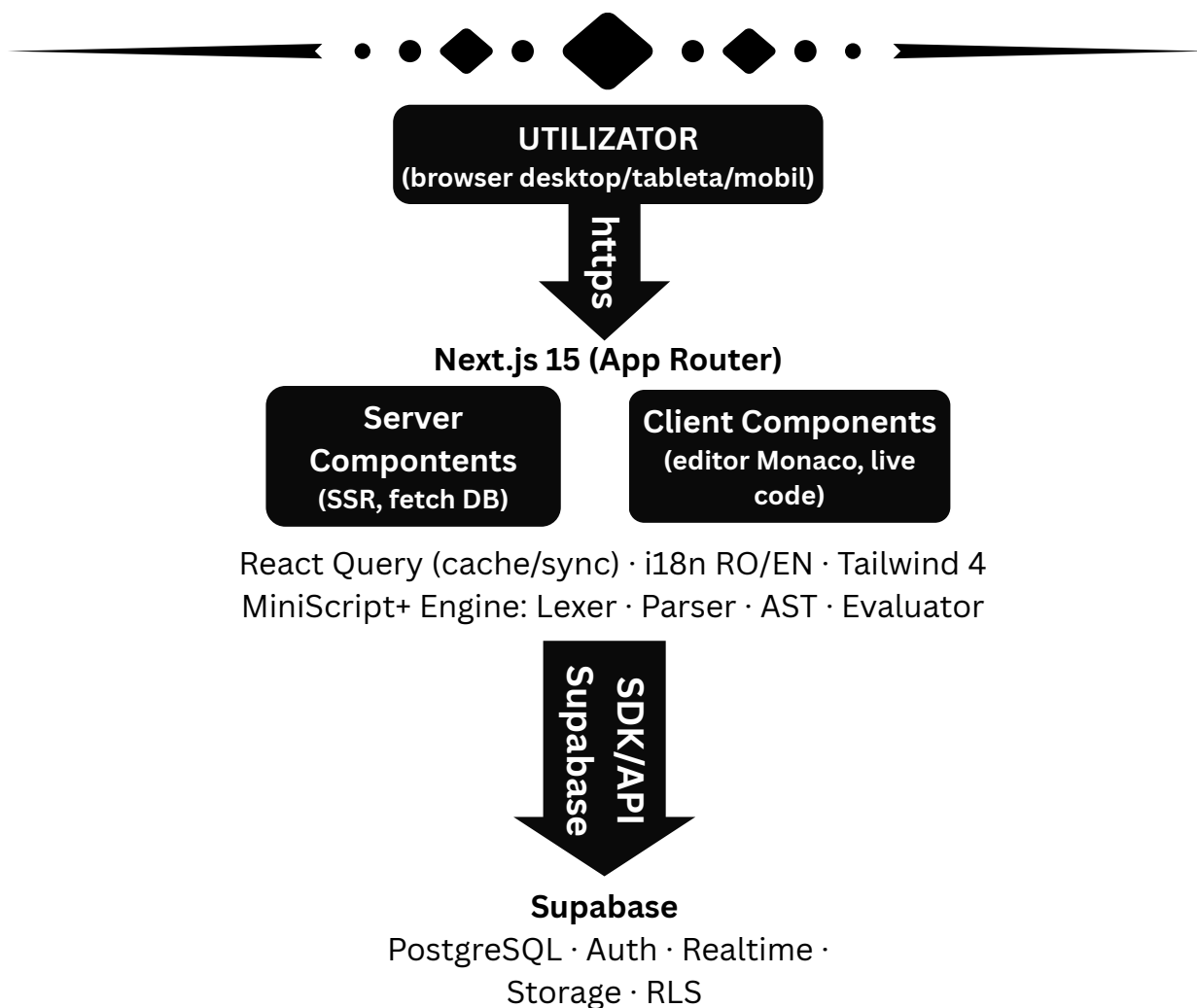
ScripticX încurajează autonomia, gândirea critică și perseverența, transformând rezolvarea problemelor într-un proces interactiv și vizibil.

Proiectul se încadrează în secțiunea Tehnologia informației, deoarece reprezintă o aplicație software funcțională, bazată pe realizare practică și demonstrație. ScripticX contribuie la dezvoltarea competențelor de cercetare, inovare, tehnologie digitală și gândire critică, obiective menționate în regulamentul ONCS.

IV. Arhitectura platformei ScripticX

ScripticX este construită pe o arhitectură modulară, de tip client-server, în care interfața web, engine-ul MiniScript+, sistemul de evaluare, baza de date și serviciile realtime sunt separate în componente independente. Această organizare permite extinderea platformei, reutilizarea componentelor și menținerea unui cod ușor de testat și dezvoltat.

Frontend-ul gestionează experiența utilizatorului, editorul de cod, dashboard-ul, problemele, sesiunile live și interacțiunile sociale. Backend-ul este asigurat prin Supabase, care oferă autentificare, bază de date PostgreSQL, politici de securitate RLS și sincronizare realtime.



Platforma este organizată pe module distincte: interfață, logică de aplicație, engine MiniScript+ și infrastructură de date. Această structură permite testarea independentă a componentelor, extinderea funcționalităților și menținerea unui flux clar între utilizator, editor, evaluator și baza de date.

V. Tehnologii utilizate

ScripticX utilizează tehnologii web moderne, alese pentru performanță, scalabilitate și ușurința extinderii. Platforma combină un frontend construit cu Next.js și React, un backend gestionat prin Supabase și un engine propriu pentru interpretarea limbajului MiniScript+. Alegerea acestor tehnologii permite rularea aplicației direct în browser, sincronizarea datelor în timp real și dezvoltarea modulară a funcționalităților.



Next.js 15 + React

Asigură structura aplicației, rutarea paginilor și separarea între componente server și componente client. Permite construirea unei aplicații rapide, responsive și ușor de extins.



TypeScript

Oferă tipare stricte pentru componente, date și motorul MiniScript+, reducând riscul erorilor și crescând lizibilitatea codului.



Tailwind CSS + shadcn/ui

Sunt folosite pentru construirea unei interfețe moderne, consistente și responsive, cu componente reutilizabile precum dialoguri, meniuri, calendare, popover-uri și breadcrumbs.



Supabase

Oferă autentificare, bază de date PostgreSQL, Storage, Realtime și politici RLS. Este folosit pentru utilizatori, probleme, sesiuni live, notificări, snippets și progres.



Monaco Editor

Este editorul de cod folosit în platformă, oferind o experiență apropiată de IDE-uri moderne: numerotare linii, highlight, configurare tab size și evidențierea liniei executate.



React Query

Optimizează încărcarea datelor prin cache, refetch controlat și sincronizare eficientă între pagini, reducând request-urile inutile către baza de date.

VI. Limbajul MiniScript+ și engine-ul

MiniScript+ este un limbaj educațional propriu, creat pentru a reduce complexitatea sintaxei și pentru a ajuta elevii să se concentreze pe logica algoritmică. Limbajul folosește instrucțiuni simple, apropiate de pseudocod, precum **INPUT**, **PRINT**, **IF**, **WHILE** și **END**, fiind potrivit pentru începători. Engine-ul MiniScript+ interpretează codul direct în browser, fără instalări suplimentare. Codul sursă este procesat printr-un flux asemănător cu cel al limbajelor reale de programare: tokenizare, parsare, construire AST și evaluare.



Evaluator + Debugger

Engine-ul parcurge instrucțiunile programului pas cu pas, actualizând variabilele, linia curentă și output-ul. În paralel, debugger-ul afișează starea execuției, iar sistemul poate evidenția linia rulată, detecta bucle infinite și estima complexitatea algoritmului.

Lexer + Parser

Codul MiniScript+ este citit și transformat în token-uri: variabile, numere, operatori, cuvinte cheie și delimitatori. Apoi parserul verifică structura instrucțiunilor și construiește reprezentarea internă a programului, identificând blocuri precum IF, WHILE, PRINT și expresii aritmetice.

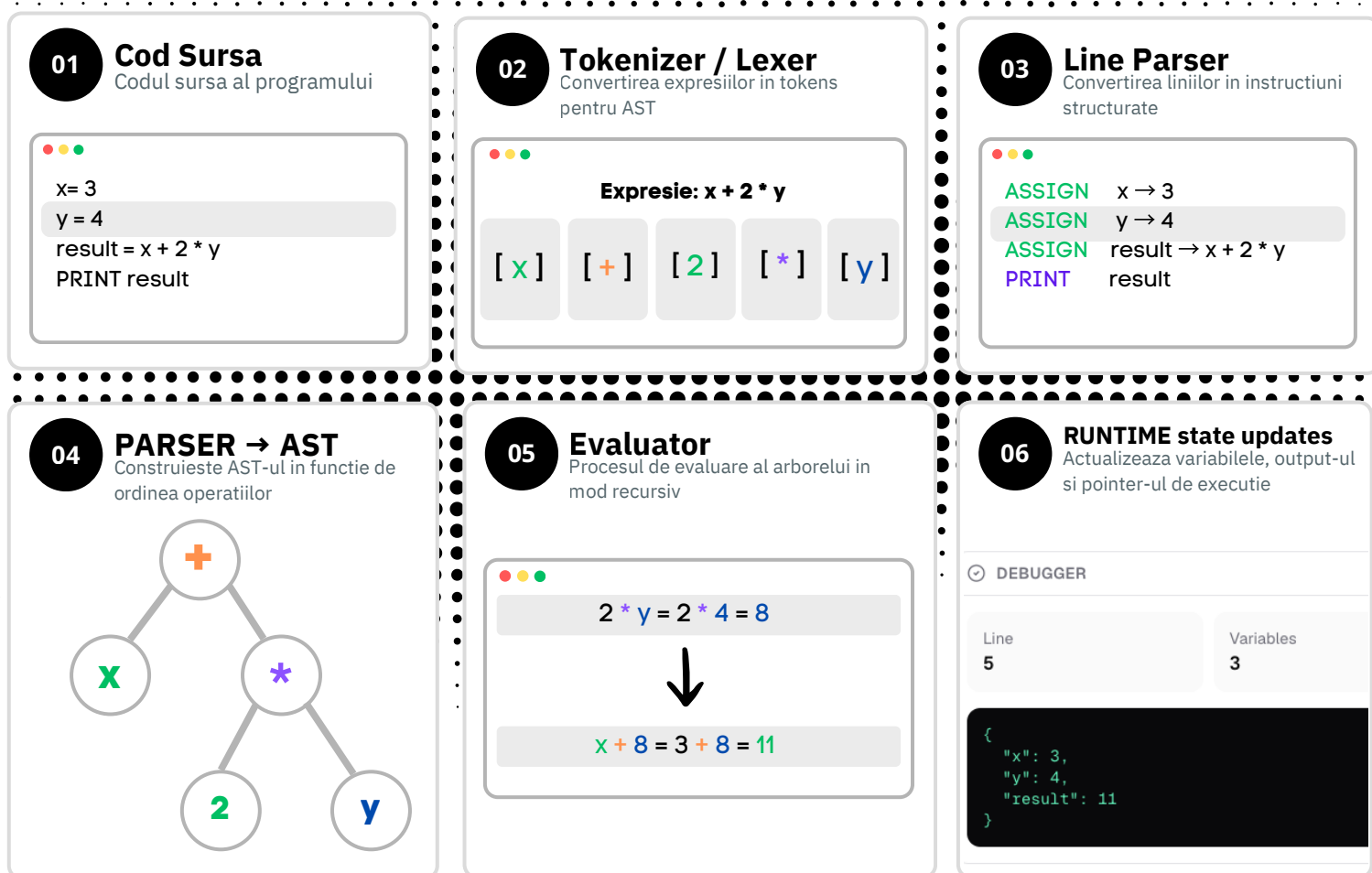
AST - Arbore Sintactic Abstract

Expresiile sunt reprezentate sub forma unui arbore sintactic abstract. Nodurile interne descriu operatori sau structuri logice, iar frunzele descriu valori, variabile și constante. AST-ul permite evaluarea expresiilor într-un mod clar, recursiv și ușor de vizualizat.

VI. Limbajul MiniScript+ și engine-ul

Engine-ul MiniScript+ interpretează codul direct în browser, transformând textul sursă în instrucțiuni structurate, AST-uri pentru expresii și execuție pas cu pas. În timpul runtime-ului, evaluatorul actualizează variabilele, output-ul și linia curentă, oferind suport pentru debugging și vizualizarea procesului intern.

EXEMPLU DE RUNTIME



Pentru o demonstrație matematică a engine-ului MS+ și o analiză a complexității implementării, puteți consulta broșura care conține modelul matematic al acestuia.

VI. Limbajul MiniScript+ și engine-ul

Analizatorul de complexitate estimează eficiența algoritmului scris în MiniScript+ fără a executa programul pentru toate valorile posibile de input. Acesta inspectează structura codului, identifică buclele, nivelul de imbricare, variabilele și instrucțiunile repetitive, apoi generează o aproximare Big-O pentru timp și spațiu. Scopul este educațional, astfel elevul înțelege de ce o soluție este mai eficientă sau mai costisitoare.

ANALIZATORUL DE COMPLEXITATE

01

Exemplu de cod

Codul sursa al programului

```
n = 5
i = 0
WHILE i < n
  PRINT i
  i = i + 1
END
```

02

Watcher

Preview pentru variabile, output și linia de cod curentă

DEBUGGER

Line
6

Variables
2

```
{
  "n": 5,
  "i": 5
}
```

03

Analiza Complexitatii

Analiză structurală a codului și estimare Big-O

COMPLEXITY ANALYZER



Time
 $O(n)$

Space
 $O(1)$

Loops
1

Max nesting
1

Suggestions

- The AST contains one main loop. The solution looks efficient for simple traversals.
- Estimated complexity is $O(n)$. This is an AST-based approximation, not a mathematical proof.

VII. Funcționalitățile platformei pentru învățare și rezolvare

ScripticX acoperă întregul proces de învățare algoritmică: scrierea codului, rularea, observarea execuției, testarea soluțiilor și primirea feedback-ului. Funcționalitățile sunt organizate modular, astfel încât elevul să poată trece natural de la experimentare liberă la rezolvarea de probleme și monitorizarea progresului.

Editor MiniScript+

Editor bazat pe Monaco, cu evidențiere de sintaxă, numerotare linii, configurare tab size, rulare instant și salvare în snippets multi-fișier.

Debugger pas cu pas

Permite urmărirea execuției linie cu linie, afișând variabilele, output-ul, linia curentă și starea internă a programului. Cunocut si drept **"Watcher"**.

Evaluator de probleme

Verifică automat soluțiile pe test case-uri, compară output-ul generat cu cel așteptat și oferă feedback imediat.

Analizator Complexitate

Estimează complexitatea în timp și spațiu, detectând bucle, imbricări și variabile relevante.

AST & Flowchart Visualizer

Transformă codul în reprezentări vizuale, ajutând elevul să înțeleagă structura logică a programului.

Daily Code Challenges

Propune provocări zilnice programate din panoul de administrare, cu punctaj separat și notificări pentru utilizatori.

Lectii despre Algoritmica

Platforma ofera lectii dedicate incepatorilor pentru a se familiariza cu sintaxa si pentru a pune la dispozitie o introducere in algoritmica.

VII. Funcționalitățile platformei sociale, colaborative și administrative

Pe lângă zona de rezolvare individuală, platforma include funcționalități de colaborare, comunitate și administrare. Aceste module transformă ScripticX într-un mediu educațional complet, nu doar într-un editor de cod.

Live Code Sessions

Sesiuni de programare colaborativă în timp real, cu participanți, chat, sincronizare live a codului, debugger și posibilitatea de gestionare a membrilor.

Multi-files Snippets

Utilizatorii pot salva fragmente de cod sub forma unor proiecte cu mai multe fișiere, publice sau private.

Feed Social

Permite publicarea de postări, cod evidențiat, comentarii, like-uri și interacțiuni între utilizatori.

Profiluri publice

Fiecare utilizator are un profil cu statistici, urmăritori, probleme rezolvate, activitate și contribuții.

Leaderboard & Achievements

Sistem de progres și gamificare bazat pe puncte, rezolvări, realizări și activitate în platformă.

Panou de administrare

Administratorii pot gestiona probleme, update-uri, utilizatori, daily challenges și conținut educațional.

Notificări

Sistem de notificări în aplicație și browser pentru invitații, provocări zilnice, interacțiuni sociale și activitate relevantă.

Suport pentru RO/EN

Interfața și conținutul educațional sunt disponibile în română și engleză, susținând internaționalizarea platformei.

VIII. Evaluare, feedback și elemente inovative

Evaluare Automată

ScripticX verifică soluțiile prin rularea codului pe test case-uri definite pentru fiecare problemă. Platforma compară output-ul generat cu output-ul așteptat și afișează imediat verdictul pentru fiecare test.

Test case-uri automate

Fiecare soluție este rulată pe date de intrare diferite, iar rezultatul este comparat cu output-ul corect.

Punctaj instant

Utilizatorul primește un scor imediat, în funcție de numărul de teste trecute.

Validare continuă

Soluțiile pot fi retrimise, permițând elevului să corecteze și să îmbunătățească algoritmul.

Feedback Educațional

Feedback-ul nu se limitează la „corect/greșit”, ci oferă informații utile despre execuție și procesul de rezolvare.

Output vizibil

Elevul poate vedea ce a produs programul și poate compara rezultatul cu cel așteptat.

Debugger pas cu pas

Platforma afișează linia curentă, variabilele și starea programului în timpul execuției.

Skeleton de evaluare

În timpul verificării, utilizatorul vede progresul testelor, de exemplu „Test 1/5 - passed”.

IX. Elemente inovative

ScripticX verifică soluțiile prin rularea codului pe test case-uri definite pentru fiecare problemă. Platforma compară output-ul generat cu output-ul așteptat și afișează imediat verdictul pentru fiecare test.

MiniScript+ Engine

Limbaj propriu interpretat direct în browser, cu lexer, parser, AST și evaluator.

AST & Flowchart Visualizer

Codul poate fi reprezentat vizual, ajutând elevul să înțeleagă structura logică a programului.

Live Code & multi-file Snippets

Elevii pot colabora în timp real și pot salva proiecte MiniScript+ cu mai multe fișiere.

Analiză de complexitate

Platforma estimează Big-O pentru timp și spațiu, evidențiind buclele, imbricările și impactul lor asupra eficienței.

Despre evaluare

Soluțiile sunt testate automat pe mai multe cazuri de test, fiecare primind verdict PASSED sau FAILED. Scorul se calculează folosind formula $S = 100 * (\text{test_cases_passed} / \text{total_tests})$. Feedback-ul include erori, output-ul obținut și suport pentru debug pas cu pas.

0 puncte

Codul tău a obținut un punctaj parțial. Analizează exemplele, încearcă să îți corectezi soluția și trimite o nouă soluție.

Test #1

Răspuns greșit

Programul a citit

3 4

Programul a afișat

-1

Programul ar fi trebuit să afișeze

7

Analizor de complexitate

100%
EXCELENT

Timp
O(1)

Spațiu
O(1)

Bucle
0

Imbricare
maximă
0

Sugestii

- AST-ul nu conține bucle, deci complexitatea estimată este foarte bună.
- Complexitatea estimată este O(1). Este o aproximare bazată pe AST, nu o dovadă matematică.

X. Testare, securitate și maturitate

ScripticX a fost dezvoltat cu accent pe stabilitate, siguranță și posibilitatea de a fi utilizat ca platformă reală. Proiectul include testare automată pentru componentele critice, politici de securitate la nivel de bază de date și o arhitectură pregătită pentru extindere.

Testare automată

Platforma folosește teste unitare pentru componentele esențiale ale engine-ului MiniScript+: lexer, parser, evaluator, AST visualizer și analizator de complexitate. Testele verifică interpretarea corectă a codului, evaluarea expresiilor, detectarea buclelor și estimarea Big-O.

Vitest: parser • engine • AST • complexity analyzer

Securitate

Autentificarea este gestionată prin Supabase Auth, iar accesul la date este controlat prin politici RLS. Datele utilizatorilor, snippets, submissions, notificările și sesiunile live sunt protejate astfel încât fiecare utilizator să poată accesa doar resursele permise.

Auth • RLS • roluri admin • acces per utilizator

Maturitatea aplicației

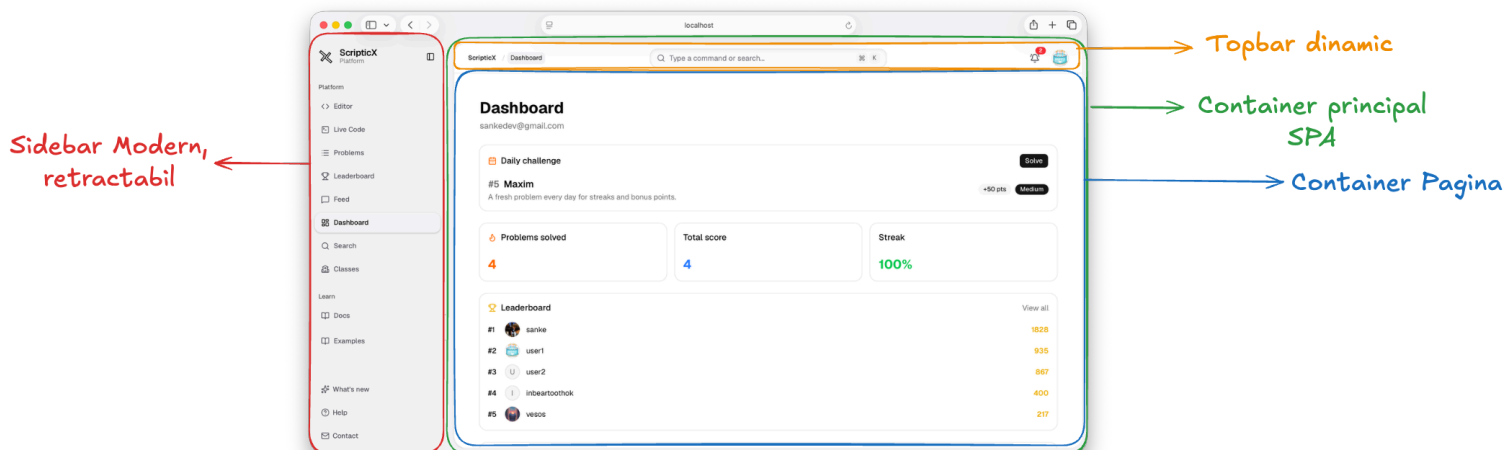
ScripticX se află într-un stadiu funcțional, cu module integrate pentru editor, probleme, evaluare automată, live coding, profiluri, feed social, notificări, administrare și integrare parțială cu GitHub. În prezent, integrarea GitHub permite conectarea și pregătirea fluxului de lucru cu repository-uri, însă nu include încă operații avansate precum push requests sau sincronizare completă bidirecțională.

Platforma poate fi rulată direct în browser, este responsive și are o arhitectură modulară, pregătită pentru următorul nivel de dezvoltare: import/export de proiecte GitHub, versionare a soluțiilor, colaborare pe branches, sistem de clase avansat, rapoarte de progres pentru profesori, badge-uri personalizate și module suplimentare de analiză educațională.

MVP funcțional • responsive • modular • pregătit pentru extindere

XI. Interfața UI/UX

Interfața ScripticX este construită pe un model de tip application shell, asemănător aplicațiilor moderne din ecosistemul Apple, GitLab, Canva sau VS Code. Structura este împărțită în zone stabile: sidebar pentru navigație, topbar pentru acțiuni globale și breadcrumb, container principal pentru aplicație și container dedicat paginii active.



Această organizare oferă utilizatorului o experiență predictibilă: meniul rămâne în aceeași poziție, acțiunile importante sunt accesibile rapid, iar conținutul se schimbă fără a rupe contextul vizual. Astfel, elevul se poate concentra pe activitatea principală, cum ar fi scrierea codului sau rezolvarea problemelor.

Sidebar modern, retractabil

Asigură navigație constantă între module și reduce spațiul ocupat atunci când utilizatorul are nevoie de mai multă zonă de lucru.

Topbar dinamic

Include breadcrumb, command menu, notificări și acțiuni globale, păstrând accesul rapid la funcțiile importante.

Container principal de tip SPA

Menține senzația unei aplicații fluide, în care paginile se schimbă rapid, fără întreruperea experienței utilizatorului.

Container pagină

Izolează conținutul fiecărei pagini, păstrând layout-ul curat, coerent și ușor de adaptat pentru desktop, tabletă și mobil.

Prin această structură vizuală, ScripticX îi ajută pe utilizatori să se orienteze ușor în platformă și oferă o experiență clară, familiară și potrivită pentru învățarea programării.

XII. Etapele de cercetare și dezvoltare

01

ANALIZA PROBLEMEI EDUCAȚIONALE

Au fost identificate dificultățile principale: sintaxa complexă, lipsa feedback-ului explicativ, dependența de soluții generate automat și nevoia unui mediu mai vizual pentru înțelegerea algoritmilor.

02

DEFINIREA LIMBAJULUI MINIScript+

A fost proiectat un limbaj educațional simplificat, apropiat de pseudocod, cu instrucțiuni de bază precum **INPUT**, **PRINT**, **IF**, **WHILE**, variabile și expresii aritmetice.

03

IMPLEMENTAREA ENGINE-ULUI

A fost construit motorul MiniScript+ cu etape clare: lexer, parser, AST și evaluator. Acesta permite rularea codului direct în browser și execuția pas cu pas.

04

DEZVOLTAREA EVALUATORULUI AUTOMAT

Au fost implementate probleme cu test case-uri, verificare automată a output-ului, punctaj și feedback imediat pentru soluțiile trimise.

05

CONSTRUIREA INTERFEȚEI PLATFORMEI

După validarea primei versiuni de UI, platforma a fost reproiectată folosind un application shell modern, cu sidebar, topbar dinamic și containere clare pentru pagini. Această structură a permis integrarea coerentă a modulelor principale: editor, dashboard, probleme, live code, profiluri, feed, notificări și suport RO/EN.

06

ADĂUGAREA FUNCȚIONALITĂȚILOR AVANSATE

Platforma a fost extinsă cu AST/flowchart visualizer, analizator de complexitate, snippets multi-fișier, daily code challenges, live coding și sistem de notificări.

07

TESTARE, OPTIMIZARE ȘI SECURIZARE

Au fost adăugate teste unitare pentru componentele critice, optimizări pentru încărcarea datelor, politici RLS în Supabase și verificări pentru stabilitatea aplicației.

08

DIRECȚII DE DEZVOLTARE VIITOARE

Următoarele etape vizează integrarea GitHub mai avansată, versionarea soluțiilor, rapoarte de progres pentru profesori, colaborare pe proiecte și module educaționale suplimentare.

XIII. Extensii viitoare și direcții de dezvoltare

ScripticX este construit modular, astfel încât platforma poate fi extinsă cu funcționalități noi fără a rescrie arhitectura existentă. Următoarele direcții de dezvoltare vizează integrarea mai profundă cu instrumente reale de programare, îmbunătățirea gestionării claselor și oferirea unor mecanisme mai avansate de urmărire a progresului.

Integrare GitHub avansată

Conectarea repository-urilor, importul și exportul proiectelor MiniScript+, versionarea soluțiilor și sincronizarea codului cu GitHub. În viitor, pot fi adăugate operații precum push, pull requests și colaborare pe branches.

Sistem de lecții interactive

Lecții structurate cu exemple, exerciții ghidate, execuție pas cu pas și mini-testări integrate după fiecare concept.

Clase avansate pentru profesori

Dashboard dedicat profesorilor, monitorizarea progresului individual, distribuirea de teme, feedback pe soluții și statistici pentru fiecare elev.

Versionarea soluțiilor

Păstrarea istoricului încercărilor pentru fiecare problemă, astfel încât elevul să poată compara soluțiile și să observe cum și-a îmbunătățit algoritmul.

Rapoarte de progres

Generarea de rapoarte pe baza problemelor rezolvate, scorului, streak-ului, daily challenges și nivelului de dificultate abordat.

Extinderea limbajului MiniScript+

Sesiuni live cu roluri diferite: profesor, elev, observator, plus posibilitatea de a urmări codul elevilor în timp real.



XIV. Componenta echipei

Distribuirea rolurilor în cadrul echipei

Lascu Andrei

Responsabil dezvoltare tehnică

- proiectarea și implementarea engine-ului MiniScript+
- dezvoltarea parserului, AST-ului și evaluatorului
- implementarea editorului, sistemului de execuție pas cu pas și analizatorului de complexitate
- integrarea funcționalităților platformei: probleme, live code, snippets, notificări și administrare
- optimizare, debugging și testare tehnică

Bobotan Andreea-Oana

Responsabil cercetare, design și documentare

- analiza problemei educaționale și definirea conceptului platformei
- proiectarea structurii vizuale și a experienței utilizatorului
- realizarea materialelor de prezentare și documentare
- contribuție la testarea funcționalităților și validarea fluxurilor educaționale
- structurarea conținutului pentru broșură, poster și prezentare

XIV. Componenta echipei

Colaborarea în echipă

Proiectul ScripticX a fost dezvoltat printr-un proces colaborativ, în care fiecare membru a contribuit la definirea conceptului, realizarea platformei și validarea funcționalităților.

Dezvoltarea a fost realizată iterativ, prin etape succesive:

- analiza problemei educaționale
- proiectarea arhitecturii platformei
- implementarea și testarea funcționalităților
- ajustarea interfeței și îmbunătățirea experienței utilizatorului
- documentarea și pregătirea prezentării proiectului

Sarcinile au fost împărțite în funcție de competențele fiecărui membru, iar deciziile importante au fost luate prin consultare și feedback constant.

Detalii echipă de proiect

Mentor
prof. Pruş Paul

Secțiune: C. Tehnologia informației

Colegiul Național „Gheorghe Munteanu Murgoci”, Brăila

Coordonare, îndrumare metodologică și validarea direcției educaționale a proiectului.

XV. Link-uri utile

github.com/Sank34/scripticx



COD SURSA PLATFORMA

github.com/Sank34/scripticx-landing



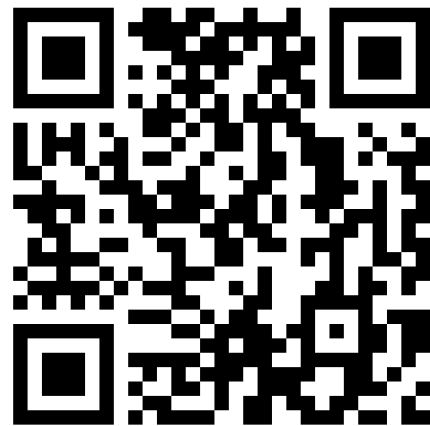
COD SURSA LANDING PAGE

scripticx.org



LANDING PAGE

platform.scripticx.org



PLATFORM